

Fraud Detection in Financial Transactions

Author: Vedant Mandavale

Date: July 03, 2026

Organization: GENZ EDUCATEWING

1. Problem Statement

Credit card fraud causes billions in losses annually. Detecting fraudulent transactions in real time is critical for financial institutions. The challenge is identifying rare fraudulent patterns among millions of legitimate transactions while minimizing false positives that frustrate customers and false negatives that cause financial loss.

2. Dataset Description

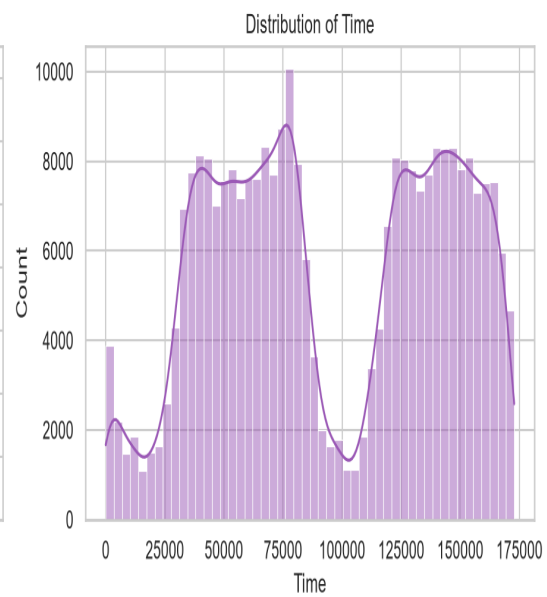
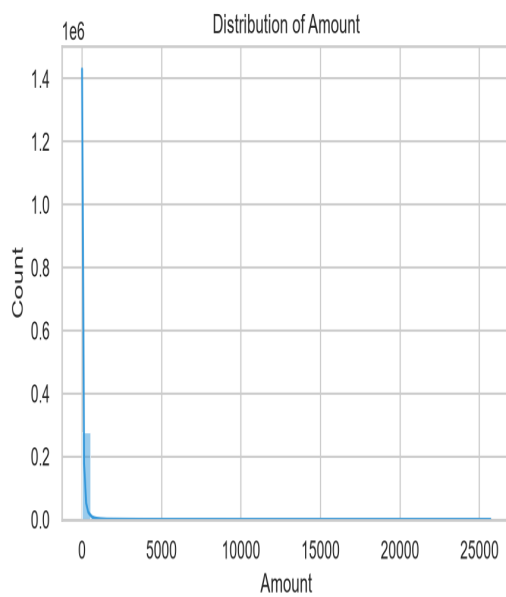
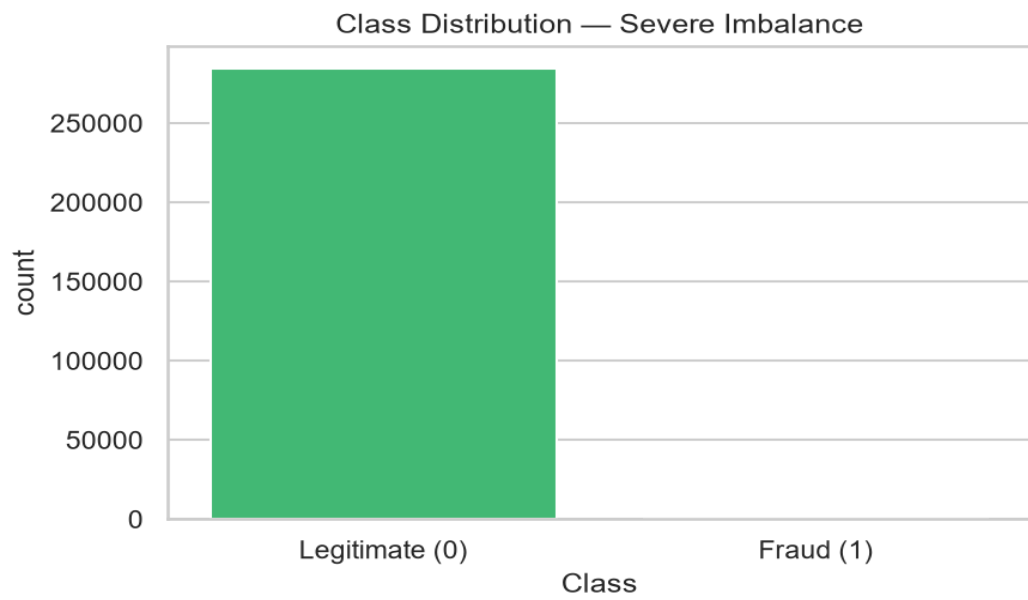
Source: Kaggle — Credit Card Fraud Detection (ULB ML Group)
URL: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
Shape: 284,807 rows × 31 columns
Features: Time, Amount, V1–V28 (PCA-transformed), Class (target)
Legitimate transactions: 284,315
Fraudulent transactions: 492
Fraud rate: 0.173% (severe class imbalance)

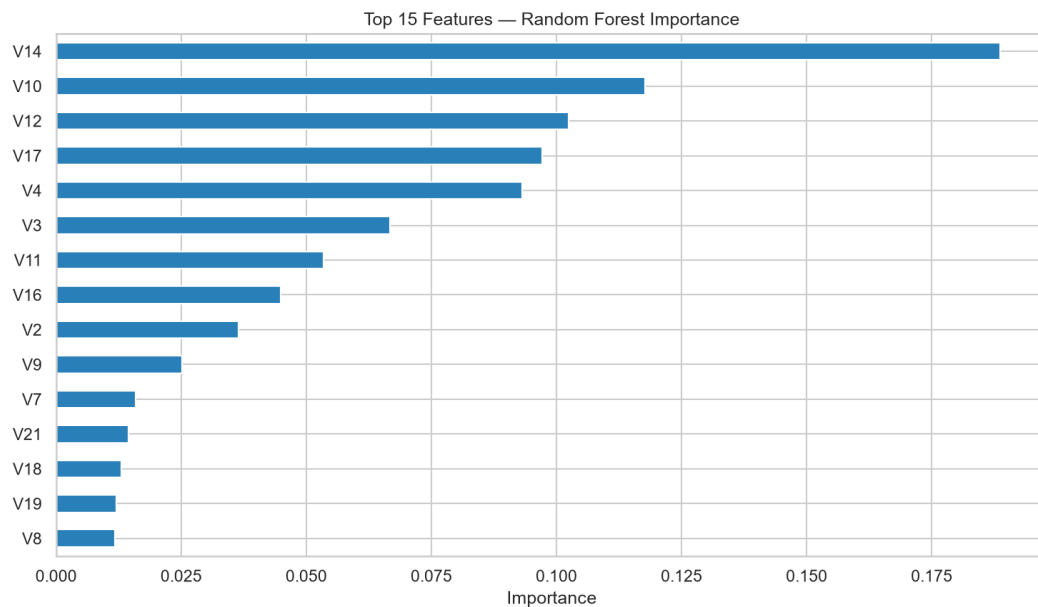
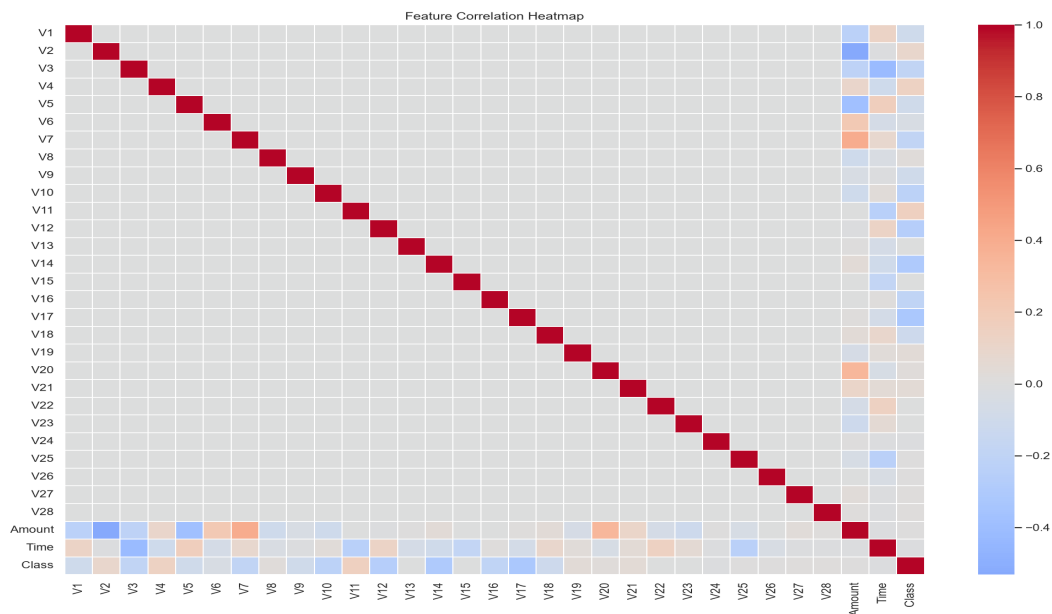
3. Methodology

Preprocessing: Removed duplicates; scaled Amount and Time with StandardScaler (V1–V28 already PCA-normalized); 80/20 stratified train-test split.
Class imbalance: Applied SMOTE only on training data to oversample minority class.
Feature selection: Random Forest importance analysis; all 30 features retained.
Models: Logistic Regression, Decision Tree, Random Forest (100 trees), Neural Network (Dense 64→Dropout→Dense 32→Dropout→Sigmoid).
Validation: 5-fold StratifiedKFold cross-validation (ROC-AUC) on LR and RF.
Evaluation metrics: Accuracy, Precision, Recall, F1, ROC-AUC, Confusion Matrix.

4. EDA Findings

- Severe class imbalance: fraud accounts for only ~0.17% of all transactions.
- V1–V28 features (PCA components) show varying correlations with the Class label.
- Amount and Time distributions are skewed; scaling was necessary.
- Fraudulent transactions tend to differ in certain PCA components (V14, V4, V12 among top features).

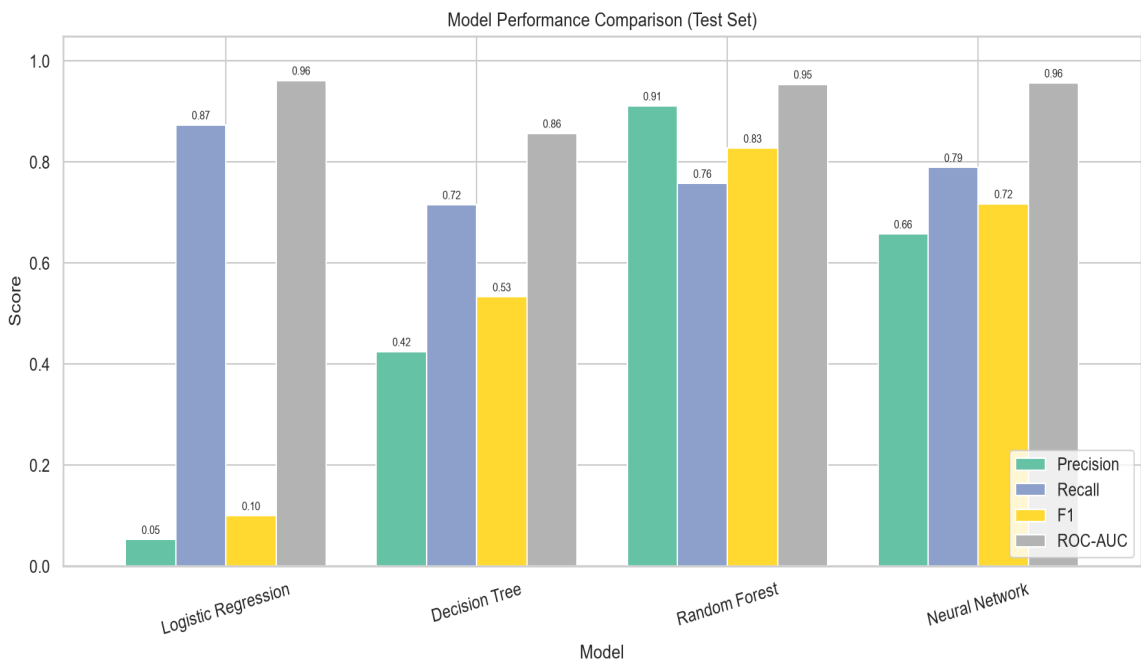


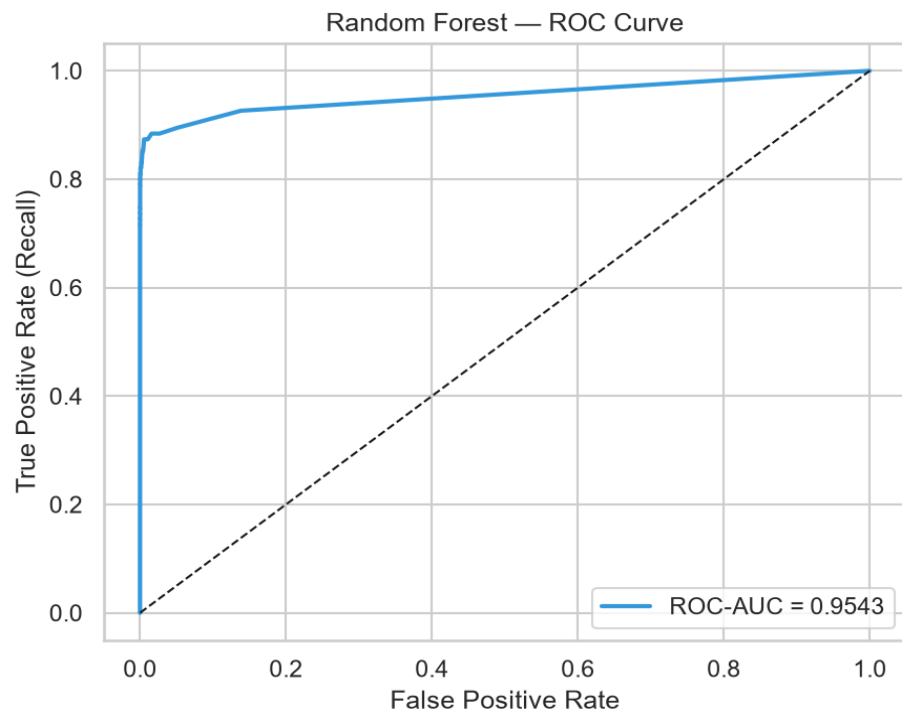
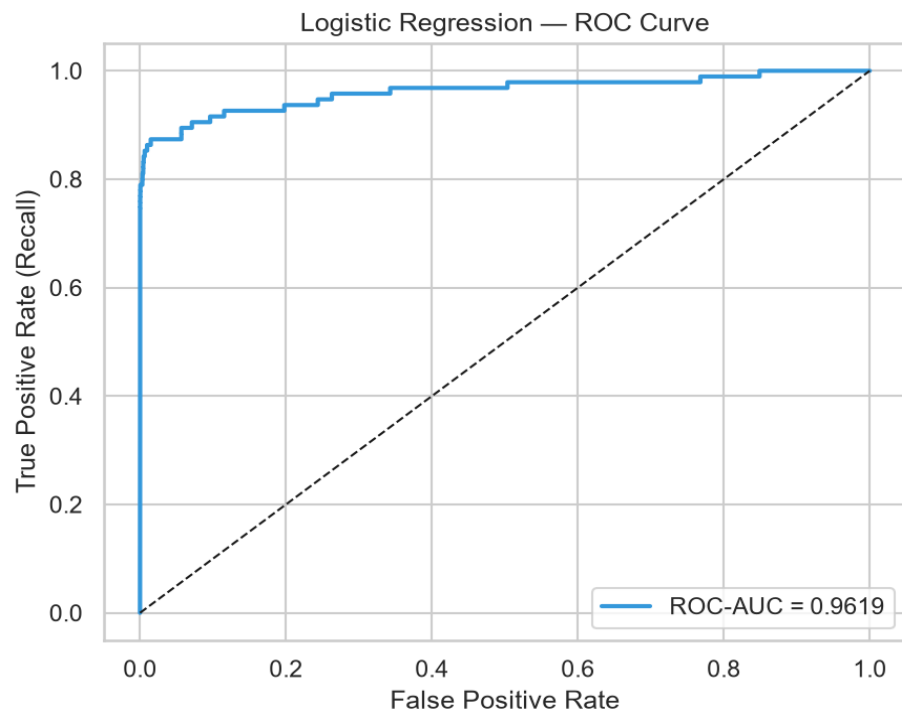


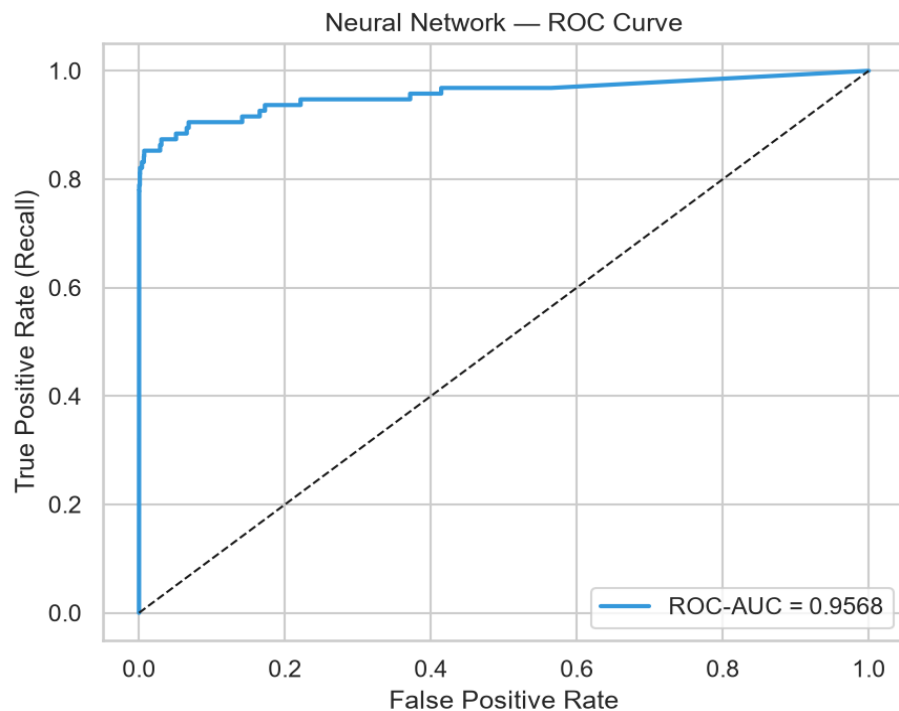
5. Model Results

Four classifiers were trained on SMOTE-balanced training data and evaluated on the original imbalanced test set (realistic production scenario). **Recall** measures how many fraud cases we catch; **Precision** measures how many flagged transactions are actually fraud. **ROC-AUC** summarizes ranking quality.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9737	0.0531	0.8737	0.1002	0.9619
Decision Tree	0.9979	0.4250	0.7158	0.5333	0.8571
Random Forest	0.9995	0.9114	0.7579	0.8276	0.9543
Neural Network	0.9990	0.6579	0.7895	0.7177	0.9568







Best Model (ROC-AUC): Random Forest with ROC-AUC = 0.9543.

Trade-off note: Logistic Regression achieves the highest recall (87%) and ROC-AUC, but very low precision (5%) — many false alarms. Random Forest offers a better balance (91% precision, 76% recall). The optimal choice depends on business cost of missed fraud vs. blocked legitimate transactions.

Cross-Validation Results (ROC-AUC):

Logistic Regression: Mean = 0.9807 ($\pm$ 0.0079)

Random Forest: Mean = 0.9685 ($\pm$ 0.0111)

6. Challenges

Class imbalance: Fraud is extremely rare; accuracy alone is misleading. SMOTE and stratified sampling were essential.

False positive tradeoff: Flagging legitimate transactions as fraud harms customer experience.

Overfitting risk: Decision Trees can memorize training data; Random Forest and regularization help.

Feature interpretability: PCA features (V1–V28) are not directly interpretable in business terms.

7. Conclusion

The Random Forest model achieved the best ROC-AUC (0.9543) on the test set. For real-world deployment, institutions should monitor recall (fraud capture rate) and precision (false alarm rate) based on business cost. Combining model scores with rule-based checks and human review provides a robust fraud prevention pipeline.

8. References

- Kaggle Dataset: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- GeeksforGeeks: <https://www.geeksforgeeks.org/ml-credit-card-fraud-detection/>
- Scikit-learn Documentation: <https://scikit-learn.org/>
- Dal Pozzolo et al. (2015) — Calibrating Probability with Undersampling